Live Link Face Integration Guide

Final Evolution Lab — Phase 4: Real-Time Facial Animation

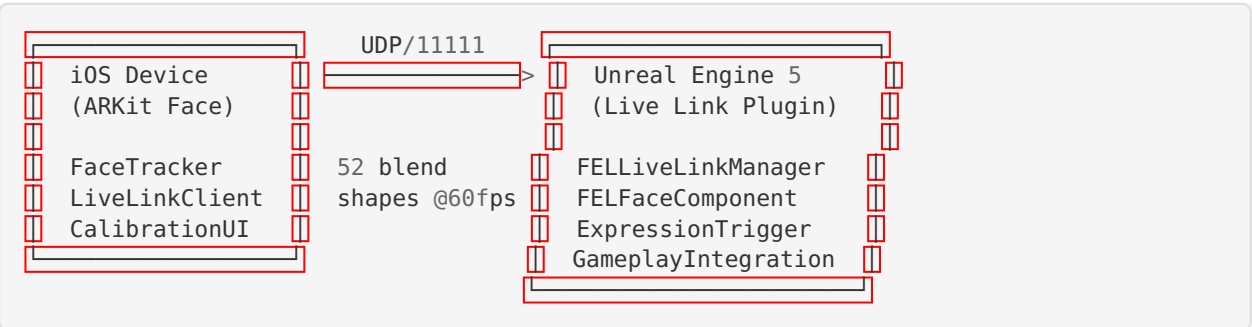
> Capture facial expressions from iPhone/iPad using ARKit and stream them to Unreal Engine in real-time via Live Link.

Table of Contents

Overview

The FEL Live Link Face system enables real-time facial animation capture from iOS devices (iPhone X or newer with TrueDepth camera) and streams 52 ARKit blend shapes to Unreal Engine 5 characters at 60 FPS.

Architecture



Key Features

- **52 ARKit Blend Shapes** — Full facial tracking (eyes, brows, jaw, mouth, cheeks, nose, tongue)
- **60 FPS Streaming** — Low-latency UDP protocol for real-time performance

- **7 Character Face Rigs** — Player, Elijah Bonds, Amir Smith, Eric Nash, 3 NPC types
- **15 Expression Presets** — Automatic fallback when Live Link is disconnected
- **20 Gameplay Triggers** — Automatic expressions for scoring, winning, exercises, etc.
- **17 Game Mode Support** — Configured for all FEL game modes
- **LOD System** — Automatic quality reduction for distant characters
- **Calibration Tool** — Neutral pose capture and range-of-motion testing

---

System Requirements

iOS Device

- iPhone X or newer (TrueDepth camera required)
- iOS 16.0 or later
- WiFi connection on same network as UE5 machine

Unreal Engine

- UE5 5.4+ with Live Link plugins enabled
- Plugins: LiveLink, LiveLinkFaceImporter, AppleARKit, AppleARKitFaceSupport

Network

- Both devices on same local network
- UDP port 11111 open
- Bonjour/mDNS for device discovery

Setup Instructions

Step 1: Enable UE5 Plugins

The `.uproject` file already includes the required plugins:

```
{"Name": "LiveLink", "Enabled": true},
{"Name": "LiveLinkFaceImporter", "Enabled": true},
{"Name": "AppleARKit", "Enabled": true},
{"Name": "AppleARKitFaceSupport", "Enabled": true}
```

Verify in UE5 Editor: **Edit → Plugins → Search “Live Link”**

Step 2: Build.cs Dependencies

The module already declares these in `FinalEvolutionLab.Build.cs`:

```
"LiveLink",
"LiveLinkInterface",
"LiveLinkComponents",
"LiveLinkAnimationCore",
"AnimGraphRuntime",
"AnimationCore"
```

Step 3: Live Link Window

In UE5 Editor:
1. **Window → Live Link**
2. Click **+ Source → Message Bus Source**
3. Your iOS device should appear when running the FEL Face Capture app

---

iOS App Installation

Requirements

- Xcode 15+ on macOS
- Apple Developer account
- iPhone X or newer

Build Steps

1. Open `MobileApps/LiveLinkFaceCapture/FELFaceCapture.xcodeproj` in Xcode
2. Set your Development Team in Signing & Capabilities
3. Set the bundle identifier to `com.finalevolutiongroup.facecapture`
4. Select your iOS device as the build target
5. Build and run (⌘R)

TestFlight Distribution

1. Archive the app (Product → Archive)
2. Upload to App Store Connect
3. Add testers in TestFlight

Connection Guide

Automatic Discovery

1. Launch UE5 project with Live Link enabled
2. Open FEL Face Capture app on iOS
3. Tap **Settings → Scan for Servers**
4. Select your UE5 machine from the list
5. Tap **Start** to begin streaming

Manual Connection

1. Find your UE5 machine's IP address
2. In the iOS app: **Settings → Server IP Address**
3. Enter the IP and port (default: 11111)
4. Tap **Start** to begin streaming

Multi-Device Setup

For multiple players, each device should use a unique Subject Name:
- Device 1: `Player1Face`
- Device 2: `Player2Face`
- etc.

Calibration Process

Why Calibrate?

Every face is different. Calibration captures your neutral resting face so the system can accurately detect expressions relative to your baseline.

Steps

1. In the iOS app, tap **Calibrate**
2. Look directly at the camera
3. Relax your face completely
4. Hold still during the 3-second countdown
5. The app captures your neutral pose
6. Test your range of motion:
   - Big smile
   - Open mouth wide
   - Raise eyebrows
   - Blink eyes
   - Puff cheeks
7. Tap **Complete Calibration**

Tips

- Recalibrate if you change lighting conditions
- Recalibrate if tracking seems off
- Keep the device ~30cm from your face
- Ensure good, even lighting on your face

Face Rigs & Characters

Character Face Rigs

Character	Type	Blend Shapes	Live Link	Signature Expressions
Player	Player	52 (full)	✅ Yes	Determined, Happy, Celebrating
Elijah Bonds	Creator	52 (full)	✅ Yes	Celebrating, TrashTalk, Determined
Amir Smith	Creator	52 (full)	✅ Yes	Determined, Angry, Pain
Eric Nash	Coach	52 (full)	✅ Yes	Happy, Determined, Surprised
NPC Athletic	NPC	32 (reduced)	❌ No	Determined, Happy, Angry
NPC Crowd	NPC	16 (minimal)	❌ No	Happy, Celebrating, Surprised
NPC Coach	NPC	32 (reduced)	❌ No	Happy, Determined, Angry

Adding Face Component to Character

In Blueprint:
1. Select your character Blueprint
2. Add Component → **FEL Live Link Face Component**
3. Set **Live Link Subject Name** (e.g., `Player1Face`)
4. Set **Character Type** (Player, ElijahBonds, etc.)
5. Configure fallback expression and LOD settings

In C++:

```
UFELLiveLinkFaceComponent* FaceComp = NewObject<UFELLiveLinkFaceComponent>(this);
FaceComp->LiveLinkSubjectName = FName("Player1Face");
FaceComp->CharacterType = EFELCharacterType::Player;
FaceComp->RegisterComponent();
```

Expression System

15 Expression Presets

Expression	Use Case	Key Blend Shapes
Neutral	Default/idle	All zeros
Happy	Positive moments	Smile + cheek squint
Determined	Active gameplay	Brow down + jaw forward
Tired	Fatigue	Half-blink + frown
Celebrating	Scoring/winning	Full smile + open mouth + wide eyes
Disappointed	Missing/losing	Frown + brow up + look down
Angry	Blocked/fouled	Brow down + nose sneer
Surprised	Unexpected plays	Wide eyes + brow up + open jaw
Pain	Max effort	Brow squeeze + nose sneer
Trash Talk	Taunting	Asymmetric smirk
Recovery	Post-exercise	Relaxed smile + open jaw
Talking A/E/O/U	Dialogue lip sync	Phoneme-specific mouth shapes

Triggering Expressions from Gameplay

```
// Get the gameplay integration subsystem
auto* FaceGameplay = GetGameInstance()->GetSubsystem<UFELFaceGameplayIntegration>();

// Automatic triggers
FaceGameplay->OnPlayerScored();  // → Celebrating expression
FaceGameplay->OnPlayerMissed();  // → Disappointed expression
FaceGameplay->OnMaxEffort();     // → Pain expression
FaceGameplay->OnGameWon();       // → Celebrating + Crowd celebration

// Manual expression control
auto* FaceComp = Character->FindComponentByClass<UFELLiveLinkFaceComponent>();
FaceComp->SetExpression(EFELExpression::TrashTalk, 0.2f);  // 0.2s blend in
FaceComp->ClearForcedExpression(0.5f);  // 0.5s blend out
```

Gameplay Integration

17 Game Mode Configurations

Each game mode has tailored face animation behavior:

Mode	Idle Expression	Active Expression	NPC Faces	Intensity
Basketball H2H	Determined	Determined	Yes (4)	1.0x
Basketball 3v3	Determined	Determined	Yes (6)	1.0x
Boxing H2H	Angry	Angry	Yes (2)	1.3x
Soccer	Determined	Determined	Yes (6)	1.0x
Workout	Neutral	Pain	No	1.0x
Track & Field	Determined	Pain	Yes (4)	1.1x
Swimming	Determined	Pain	No	1.0x

20 Gameplay Event Triggers

Event	Expression	Duration	Priority
Score Basket	Celebrating	3.0s	5
Miss Shot	Disappointed	2.0s	3
Get Blocked	Angry	1.5s	4
Win Game	Celebrating	5.0s	10
Lose Game	Disappointed	4.0s	10
Complete Workout	Recovery	3.0s	6
Max Effort	Pain	1.5s	7
Knock Out (Boxing)	Celebrating	5.0s	9
Taunt	Trash Talk	2.5s	4

Performance Optimization

LOD System

Face animation quality scales with camera distance:

LOD	Distance	Blend Shapes	Target FPS
0	0-1500	52 (full)	60
1	1500-3000	32 (reduced)	60
2	3000-5000	16 (minimal)	30
3	5000+	Disabled	—

Network Optimization

- **UDP Protocol** — Minimal overhead, no handshake
- **Compressed Packets** — 288 bytes per frame (52 floats + header)
- **Rate Limiting** — Capped at 60 FPS send rate
- **Priority System** — Local player always gets full quality

Memory Budget

- Full face rig (52 shapes): ~2.1 KB per frame
- Reduced rig (32 shapes): ~1.3 KB per frame
- Minimal rig (16 shapes): ~0.7 KB per frame
- Expression presets: ~4 KB total (static)

Config Tuning (`DefaultLiveLink.ini`)

```
; Reduce for better performance on low-end devices
TargetFPS=30
ReducedFPSDistance=1000.0
DisableDistance=3000.0
MaxSimultaneousConnections=4

; Smoothing (higher = smoother but more latent)
DefaultSmoothingAlpha=0.15
EnableOneEuroFilter=True
```

Troubleshooting

iOS App Can’t Find UE5 Server

1. Verify both devices are on the same WiFi network
2. Check firewall allows UDP port 11111
3. Try manual IP connection instead of discovery
4. Restart the Live Link window in UE5

Face Tracking Not Working

1. Ensure iPhone X or newer (TrueDepth camera required)
2. Check camera permission is granted
3. Ensure adequate lighting (avoid backlighting)
4. Check device isn’t too far from face (optimal: 30-50cm)

Expressions Look Wrong

1. Run calibration (Calibrate button in app)
2. Check character has correct face rig assigned
3. Verify morph target names match ARKit convention
4. Adjust smoothing (lower = more responsive, higher = smoother)

Performance Issues

1. Reduce `MaxSimultaneousConnections` in config
2. Increase `DisableDistance` to skip distant characters
3. Lower `TargetFPS` to 30
4. Disable NPC facial animation in game mode config

Connection Drops

1. System auto-reconnects up to 5 times
2. Check WiFi signal strength
3. Reduce interference from other devices
4. Try reducing send rate to 30 FPS

Supported Devices

Device	Face Tracking	Notes
iPhone X	✅	TrueDepth camera
iPhone XS/XR	✅	Better performance
iPhone 11/12/13/14/15/16	✅	Best performance
iPad Pro (3rd gen+)	✅	TrueDepth camera
iPad Air (M1+)	❌	No TrueDepth
Android	❌	Not supported (no ARKit)

API Reference

UFELLiveLinkManager (Subsystem)

```
void StartDeviceDiscovery();
void StopDeviceDiscovery();
bool ConnectToDevice(const FFELLiveLinkDevice& Device);
void DisconnectAll();
TMap<FName, float> GetBlendShapeValues(const FName& SubjectName);
void CaptureNeutralPose(const FName& SubjectName);
void SetSmoothingAlpha(float Alpha);
void SetFaceTrackingEnabled(bool bEnabled);
```

UFELLiveLinkFaceComponent

```
void SetExpression(EFELExpression Expression, float BlendTime);
void ClearForcedExpression(float BlendTime);
void BlendWithExpression(EFELExpression Expression, float BlendAlpha, float Blend-
Time);
TMap<FName, float> GetCurrentBlendShapes();
bool IsReceivingLiveLinkData();
```

UFELExpressionTriggerSubsystem

```
void TriggerExpression(AActor* TargetActor, EFELGameplayEvent Event);
void TriggerPlayerExpression(EFELGameplayEvent Event);
void RegisterTrigger(EFELGameplayEvent Event, const FFELExpressionTrigger& Trigger);
```

UFELFaceGameplayIntegration

```
void OnGameModeStarted(const FString& GameModeId);
void OnPlayerScored();
void OnPlayerMissed();
void OnGameWon();
void OnGameLost();
void OnExerciseStarted();
void OnMaxEffort();
void OnCutsceneDialogue(AActor* Speaker);
void TriggerCrowdCelebration();
```

File Structure

```
UnrealStarter/BasketballGame/
├── FinalEvolutionLab.uproject          # Live Link plugins enabled
├── Config/
│   └── DefaultLiveLink.ini             # Network, tracking, performance settings
├── Content/FEL/
│   ├── Characters/
│   │   └── FaceRigConfigs.json         # 7 character face rig configs
│   └── Animations/FaceExpressions/
│       └── ExpressionPresets.json      # 15 expression presets + trigger map
└── Source/FinalEvolutionLab/
    ├── FELLiveLinkManager.h/.cpp       # Connection management subsystem
    ├── FELLiveLinkFaceComponent.h/.cpp # Per-character face component
    ├── FELFaceRigTypes.h               # Enums, structs, ARKit blend shapes
    ├── FELExpressionTrigger.h/.cpp     # Event → expression mapping
    └── FELFaceGameplayIntegration.h/.cpp # 17 game mode integration

MobileApps/LiveLinkFaceCapture/
└── FELFaceCapture/
    ├── FELFaceCaptureApp.swift         # App entry point
    ├── ContentView.swift               # Main UI with camera preview
    ├── FaceTracker.swift               # ARKit face tracking (52 shapes)
    ├── LiveLinkClient.swift            # UDP streaming to UE5
    ├── SettingsView.swift              # Connection & tracking settings
    ├── CalibrationView.swift           # Neutral pose calibration
    └── Info.plist                      # Camera & network permissions

Tools/
└── test_livelink_face.py               # Comprehensive test suite

docs/
└── LIVE_LINK_FACE_GUIDE.md             # This guide
```

Phase 4 of Final Evolution Lab — Built for real-time competitive gaming with facial expression capture.